

Hate Speech Classification Using Deep Learning with TensorBoard Logging

Adesijibomi Aderinto

Oghenero Jeremiah Opia

2025-06-04

Abstract

This work presents a comprehensive pipeline for hate speech detection on textual data using deep learning techniques. We implement two models one trained from scratch with an LSTM-based architecture, and another leveraging pretrained transformer-based embeddings from DistilBERT. Both models incorporate preprocessing, training, evaluation, and visualization components, with detailed logging via TensorBoard to aid interpretability and performance analysis.

I. Introduction Detecting hate speech in social media and online content (Ashwini 2020) is a critical problem requiring robust natural language processing (NLP) techniques. Traditional machine learning approaches are limited by their feature extraction capabilities. Deep learning models, especially those employing recurrent neural networks (RNNs) and pretrained transformers (Sanh et al. 2019), have shown superior performance. This study implements two distinct architectures for binary classification (hate vs. non-hate speech) (mrmorj 2022), including model training, validation, and interpretability through embedding visualizations and confusion matrices. All codes and results can be found in github repo (Adeade 2025).

II. Methods A. Data Preprocessing

Input text data undergoes extensive preprocessing to enhance model effectiveness. The pipeline converts all text to lowercase, removes emails, IP addresses, punctuation, digits, and English stop words using NLTK tools. Tokenization is

performed to facilitate subsequent vectorization.

```
def preprocess_pandas(data, text_col='review'):
    data = data.copy()
    data[text_col] = data[text_col].str.lower()
    data[text_col] = data[text_col].replace(r'([a-zA-Z0-9_]+@[a-zA-Z0-9_]+\.[a-zA-Z]{2,})', '', regex=True)
    data[text_col] = data[text_col].replace(r'([0-9]{1,3}[0-9]{0,1}[0-9]{0,1}|[0-9]{1,3})', '', regex=True)
    data[text_col] = data[text_col].replace(r'([a-zA-Z]{2,})', '', regex=True)
    data[text_col] = data[text_col].replace(r'([a-zA-Z]{2,})', '', regex=True)
    stop_words = set(stopwords.words('english'))
    data[text_col] = data[text_col].apply(lambda x: " ".join(
        [word for word in word_tokenize(x) if word not in stop_words]
    ))
    return data
```

fig.1 Data preprocessing

B. Model Architectures and Training Two training regimes are implemented: 1. *From-Scratch LSTM Model*: The first model uses a bidirectional Long Short-Term Memory (LSTM) network with an embedding layer trained from scratch. Tokenization and padding standardize input sequences to fixed lengths. The network includes dropout regularization and dense layers culminating in a sigmoid output for binary classification. Training uses binary cross-entropy loss and Adam optimization. Dataset splitting reserves 30% for validation and testing.

```

if not use_pretrained:
    model = Sequential([
        Embedding(input_dim=vocab_size, output_dim=embedding_dim, trainable=True, mask_zero=True),
        Bidirectional(LSTM(32)),
        Dense(16, activation='relu'),
        Dropout(0.3),
        Dense(1, activation='sigmoid')
    ])

    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

    history = model.fit(X_train, y_train, epochs=epochs, validation_data=(X_val, y_val), batch_size=batch_size)

    for epoch in range(epochs):
        writer.add_scalar('Loss/train', history.history['loss'][epoch], epoch)
        writer.add_scalar('Accuracy/train', history.history['accuracy'][epoch], epoch)
        writer.add_scalar('Loss/val', history.history['val_loss'][epoch], epoch)
        writer.add_scalar('Accuracy/val', history.history['val_accuracy'][epoch], epoch)

    # Evaluation
    test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
    print(f" Evaluation on test set - Loss: (test_loss:.4f), Accuracy: (test_accuracy:.4f)")

```

fig.2 LSTM Model From Scratch

2. **Pretrained DistilBERT Model:** The second model exploits the transformer-based DistilBERT, fine-tuned on the task. Dataset samples are tokenized using Hugging Face's tokenizer, with class imbalance handled via weighted cross-entropy loss. Training is conducted with Hugging Face's Trainer API, and evaluation metrics include accuracy scores on held-out test sets.

C. Logging and Visualization Both models log training and validation losses and accuracies to TensorBoard. Embedding visualizations of word and sentence embeddings are generated to facilitate interpretability. Confusion matrices are plotted to provide insight into classification errors. For the pretrained model, [CLS] token embeddings are extracted for visualization.

- III. **Results** The *from-scratch LSTM model* achieves a test accuracy of approximately 91.5%, with effective learning demonstrated by convergence of loss curves.

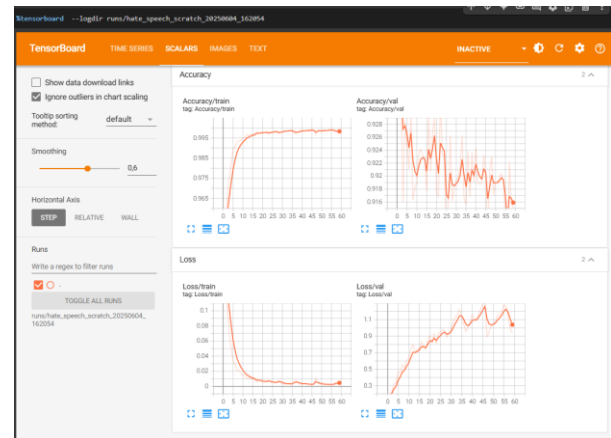


fig.3 LSTM loss and Accuracy

- Training was done for 60 epochs with accuracy of 91.5%
- Model has not converged, we can still see validation loss increasing, but for the scope of the project, we want to visualize the word and sentence embedding.
- An Embedding layer: turns tokens into dense vectors (word embedding).
- A Bidirectional(LSTM) layer: processes word embedding in both forward and backward directions to capture sentence context.
- Sentence embedding are dense vector representations of entire sentences, capturing semantic and syntactic information.

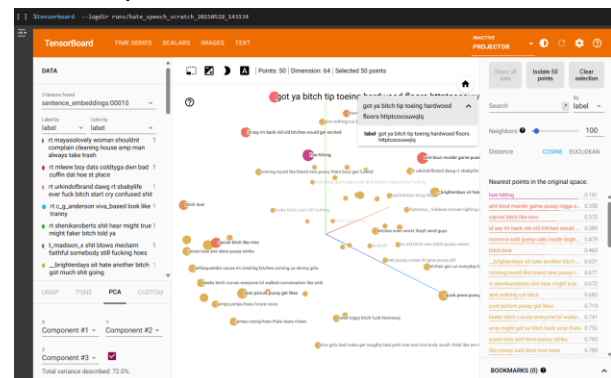


fig.4 Sentence Embeddings

DistilBERT model improves performance significantly, showcasing the benefits of transfer learning. TensorBoard visualizations reveal meaningful clustering of embeddings corresponding to semantic classes. Confusion matrices highlight classification strengths and weaknesses.

In the LSTM model (trained from scratch), embeddings are extracted from the final output of a Bidirectional LSTM. In the pretrained *DistilBERT* model, the [CLS] token representation from the transformer's output is used.

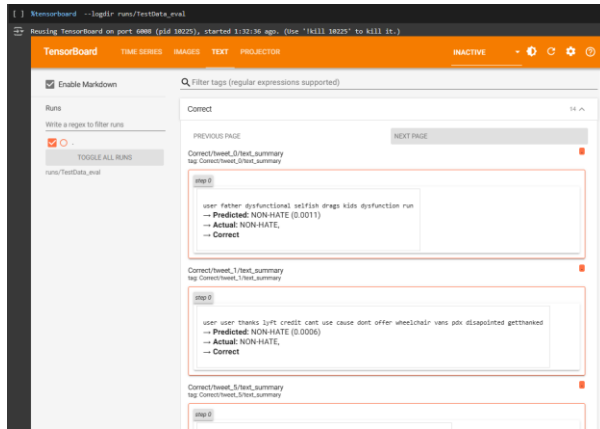


fig.6 *DistilBert Text Classification*

IV. Conclusion This study demonstrates the efficacy of deep learning for hate speech detection, highlighting the advantage of pretrained transformer embeddings over training models from scratch. The integration of TensorBoard for real-time monitoring and embedding analysis provides valuable tools for model diagnostics. Future work will explore model distillation and deployment for real-time content moderation.

V. Reference

Adeade. 2025. "Adv_DL-." https://github.com/adeade-4/Adv_DL-. https://github.com/adeade-4/Adv_DL-.

Ashwini. 2020. "Toxic Tweets Dataset." <https://www.kaggle.com/datasets/ashwiniyer176/toxic-tweets-dataset>.

mrmorj. 2022. "Hate Speech and Offensive Language Dataset." <https://www.kaggle.com/datasets/mrmorj/hate-speech-and-offensive-language-dataset>.

Sanh, Victor, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. "DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter." *arXiv Preprint arXiv:1910.01108*. <https://arxiv.org/abs/1910.01108>.